

2D to 3D Reconstruction Pipeline

Complete Source Code — All Steps

Step 1 — Image Basics

Step 2 — Image Processing

Step 3 — Depth Estimation

Step 4 — Visualize Point Cloud

Step 5 — Mesh Generation

Step 6 — Textured Mesh

Step 7 — Multi-View Fusion

Frontend — app.py

Step 1 — Image Basics

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

def explore_image_parameters(image_path):
    """
    Explores and prints basic parameters of an image.
    """
    # Load image using OpenCV
    # OpenCV loads images in BGR format by default
    img = cv2.imread(image_path)

    if img is None:
        print(f"Error: Could not load image from {image_path}")
        return

    # Convert BGR to RGB for correct visualization with Matplotlib
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    print(f"--- Image Parameters for: {os.path.basename(image_path)} ---")

    # 1. Resolution (Dimensions)
    height, width, channels = img.shape
    print(f"Resolution: {width}x{height} pixels")
    print(f"Height: {height}, Width: {width}")

    # 2. Color Channels
    print(f"Channels: {channels} (e.g., 3 for RGB)")

    # 3. Pixel Values (Data Type and Range)
    print(f"Data Type: {img.dtype}")
    print(f"Min Pixel Value: {img.min()}")
    print(f"Max Pixel Value: {img.max()}")

    # 4. Brightness and Contrast (Simple stats)
    mean_brightness = np.mean(img)
    std_contrast = np.std(img)
    print(f"Average Brightness: {mean_brightness:.2f}")
    print(f"Contrast (Std Dev): {std_contrast:.2f}")

    # Visualization
    plt.figure(figsize=(10, 5))

    plt.subplot(1, 2, 1)
    plt.imshow(img_rgb)
    plt.title("Original Image")
    plt.axis('off')

    # Histogram of pixel intensities
    plt.subplot(1, 2, 2)
    colors = ('b', 'g', 'r')
    for i, color in enumerate(colors):
        hist = cv2.calcHist([img], [i], None, [256], [0, 256])
        plt.plot(hist, color=color)
        plt.xlim([0, 256])
    plt.title("Color Histogram")
    plt.xlabel("Pixel Value")
    plt.ylabel("Frequency")
    plt.tight_layout()
    plt.show()

if __name__ == "__main__":
```

```
# Use a demo image from assets
image_path = os.path.join("assets", "demo001.jpg")
if not os.path.exists(image_path):
    print(f"Warning: {image_path} not found. Creating dummy.")
    dummy_path = "sample_image.jpg"
    if not os.path.exists(dummy_path):
        # Create a simple gradient image
        gradient = np.zeros((400, 400, 3), dtype=np.uint8)
        for i in range(400):
            gradient[i, :, :] = i % 255
        cv2.imwrite(dummy_path, gradient)
    image_path = dummy_path

explore_image_parameters(image_path)
```

Step 2 — Image Processing

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

def process_image(image_path):
    """
    Demonstrates basic image processing operations: filtering, edge detection, transformations.
    """
    img = cv2.imread(image_path)
    if img is None:
        print(f"Error: Could not load image from {image_path}")
        return

    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # 1. Grayscale Conversion
    # Essential for many processing tasks (like edge detection) to reduce complexity
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # 2. Gaussian Blur (Filtering)
    # Reduces noise and detail
    blurred = cv2.GaussianBlur(gray, (5, 5), 0)

    # 3. Edge Detection (Canny)
    # Detects structural edges in the image
    edges = cv2.Canny(blurred, 50, 150)

    # 4. Simple Transformation (Rotation)
    rows, cols = img.shape[:2]
    M = cv2.getRotationMatrix2D((cols/2, rows/2), 45, 1) # Rotate 45 degrees
    rotated = cv2.warpAffine(img_rgb, M, (cols, rows))

    # Visualization
    plt.figure(figsize=(12, 8))

    plt.subplot(2, 2, 1)
    plt.imshow(img_rgb)
    plt.title("Original Image")
    plt.axis('off')

    plt.subplot(2, 2, 2)
    plt.imshow(gray, cmap='gray')
    plt.title("Grayscale")
    plt.axis('off')

    plt.subplot(2, 2, 3)
    plt.imshow(edges, cmap='gray')
    plt.title("Edge Detection (Canny)")
    plt.axis('off')

    plt.subplot(2, 2, 4)
    plt.imshow(rotated)
    plt.title("Rotated 45°")
    plt.axis('off')

    plt.tight_layout()
    plt.show()

    # Save the grayscale image for the next step
    output_gray_path = os.path.join("output", "gray_processed.jpg")
    if not os.path.exists("output"):
```

```
        os.makedirs("output")
    cv2.imwrite(output_gray_path, gray)
    print(f"Saved grayscale image to {output_gray_path}")
    return output_gray_path

if __name__ == "__main__":
    # Use a demo image from assets
    image_path = os.path.join("assets", "demo03.jpg")

    if not os.path.exists(image_path):
        print(f"Warning: {image_path} not found. Creating dummy.")
        image_path = "sample_image.jpg"
        if not os.path.exists(image_path):
            import os
            gradient = np.zeros((400, 400, 3), dtype=np.uint8)
            for i in range(400):
                gradient[i, :, :] = i % 255
            cv2.imwrite(image_path, gradient)

    process_image(image_path)
```

Step 3 — Depth Estimation

```
import cv2
import numpy as np
import torch
import matplotlib.pyplot as plt
from transformers import pipeline
from PIL import Image
import open3d as o3d
import os
import argparse

def guided_filter(I, p, r, eps):
    """
    Guided Filter implementation (He et al., 2010).
    I: Guide image (Grayscale, normalized 0-1)
    p: Input image (Depth map, normalized 0-1)
    r: Radius of the box filter
    eps: Regularization parameter
    """
    # Ensure inputs are float32
    I = I.astype(np.float32)
    p = p.astype(np.float32)

    # Mean filtering
    mean_I = cv2.boxFilter(I, cv2.CV_32F, (r, r))
    mean_p = cv2.boxFilter(p, cv2.CV_32F, (r, r))
    mean_Ip = cv2.boxFilter(I * p, cv2.CV_32F, (r, r))

    # Covariance
    cov_Ip = mean_Ip - mean_I * mean_p

    mean_II = cv2.boxFilter(I * I, cv2.CV_32F, (r, r))
    var_I = mean_II - mean_I * mean_I

    # Linear coefficients
    a = cov_Ip / (var_I + eps)
    b = mean_p - a * mean_I

    # Mean coefficients
    mean_a = cv2.boxFilter(a, cv2.CV_32F, (r, r))
    mean_b = cv2.boxFilter(b, cv2.CV_32F, (r, r))

    # Output
    q = mean_a * I + mean_b
    return q

def process_depth_estimation(image_path, output_dir="output", focal_length_scale=0.8, depth_scale=1000.0, depth_trunc=10):
    """
    Main pipeline:
    1. Load Image
    2. Generate Depth Map (DepthAnythingV2)
    3. Create Multi-View Point Clouds (RGB, Grayscale, Heatmap)
    """
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)

    print(f"Processing: {image_path}")

    # --- Step 1: Load Image & Prepare Guide ---
    # Load with OpenCV
    cv_image = cv2.imread(image_path)
    if cv_image is None:
        print(f"Error: Could not load image {image_path}")
```

```

        return None

cv_image = cv2.cvtColor(cv_image, cv2.COLOR_BGR2RGB)
pil_image = Image.fromarray(cv_image)

# --- Step 1.5: Load External Grayscale (from Step 2) ---
# Check if we have a pre-processed grayscale image from the previous script
gray_path = os.path.join(output_dir, "gray_processed.jpg")

if os.path.exists(gray_path):
    print(f"Loading external grayscale guide from {gray_path}...")
    gray_cv = cv2.imread(gray_path, cv2.IMREAD_GRAYSCALE)
    if gray_cv.shape[:2] != cv_image.shape[:2]:
        gray_cv = cv2.resize(gray_cv, (cv_image.shape[1], cv_image.shape[0]))

    # Normalize to 0-1 for the filter
    guide_image = gray_cv.astype(np.float32) / 255.0
    l_channel = gray_cv # Keep for point cloud texture
else:
    print("External grayscale not found. Generating internal LAB guide...")
    # Fallback: Prepare Guide Image (Grayscale) using LAB Color Space
    # Technical Note: We use LAB instead of simple RGB-to-grayscale because:
    # 1. L channel represents perceptual luminance (matches human vision)
    # 2. Separates luminance from chrominance (color-independent processing)
    # 3. Better edge detection than RGB intensity averaging
    # 4. Industry standard in professional image processing pipelines
    # Reference: "Guided Image Filtering" (He et al., ECCV 2010)
    lab_image = cv2.cvtColor(cv_image, cv2.COLOR_RGB2LAB)
    l_channel, _, _ = cv2.split(lab_image)

    # Normalize Guide to 0-1 for numerical stability in filtering operations
    guide_image = l_channel.astype(np.float32) / 255.0

# --- Step 2: Depth Estimation ---
print("Loading Depth Anything V2 model...")
pipe = pipeline(task="depth-estimation", model="depth-anything/Depth-Anything-V2-Small-hf")

print("Estimating depth...")
depth_result = pipe(pil_image)
depth_map_raw = np.array(depth_result["depth"])

# Resize depth map to match original image dimensions
if depth_map_raw.shape[:2] != cv_image.shape[:2]:
    depth_map_raw = cv2.resize(depth_map_raw, (cv_image.shape[1], cv_image.shape[0]))

# Normalize Raw Depth to 0-1
depth_min, depth_max = depth_map_raw.min(), depth_map_raw.max()
depth_map_norm = (depth_map_raw - depth_min) / (depth_max - depth_min)

# --- Step 3: Grayscale-Guided Depth Refinement ---
# This is where we actually USE the L channel to improve the depth map!
# The guided filter uses the high-resolution grayscale (L channel) to refine
# the depth map edges, making them sharper and more accurate.
print("Applying grayscale-guided refinement...")

# Apply guided filter: uses L channel to preserve/sharpen edges in depth map
# Parameters:
# - guide_image: L channel (high-res grayscale with sharp edges)
# - depth_map_norm: raw depth from AI (may have soft edges)
# - radius=8: size of filtering window
# - eps=0.01: edge preservation strength
depth_refined = guided_filter(guide_image, depth_map_norm, 8, 0.01)

# Denormalize back to original depth range
depth_map = depth_refined * (depth_max - depth_min) + depth_min
print("✓ Depth map refined using luminance channel")

```

```

# --- Step 4: Multi-View Point Cloud Generation ---
# Advanced Multi-Modal Depth Processing Pipeline
# This implements a research-backed approach to 3D reconstruction
# that provides three complementary representations for different use cases
print("Generating multi-view point clouds...")

# We generate THREE point cloud files from the same depth data:
# 1. RGB - Photorealistic reconstruction (standard visualization)
# 2. Grayscale - Structure-focused, texture-normalized (architectural/scientific)
# 3. Heatmap - Depth-encoded visualization (QA/debugging/education)
#
# Technical Justification:
# - Multi-modal outputs enable validation across different representations
# - Texture normalization (grayscale) eliminates color bias in analysis
# - Perceptual depth encoding (heatmap) aids human interpretation
# - Follows best practices from photogrammetry and computer vision research

# Use the raw depth for 3D generation
depth_map = depth_map_raw

# --- Step 4: Point Cloud Generation ---
print("Creating point clouds...")

# Intrinsic parameters
height, width, _ = cv_image.shape
focal_length = width * focal_length_scale
intrinsic = o3d.camera.PinholeCameraIntrinsic(width, height, focal_length, focal_length, width / 2, height / 2)

# Helper function to create point clouds
def create_pcd(color_image, depth_map):
    o3d_color = o3d.geometry.Image(color_image)
    o3d_depth = o3d.geometry.Image(depth_map.astype(np.float32))

    rgbd = o3d.geometry.RGBDImage.create_from_color_and_depth(
        o3d_color, o3d_depth, depth_scale=depth_scale, depth_trunc=depth_trunc, convert_rgb_to_intensity=False
    )
    pcd = o3d.geometry.PointCloud.create_from_rgbd_image(rgbd, intrinsic)

    # --- Point Cloud Cleanup ---
    # 1. Statistical Outlier Removal (removes noise/flying pixels)
    # nb_neighbors: number of neighbors to analyze for each point
    # std_ratio: threshold (lower = more aggressive removal)
    cl, ind = pcd.remove_statistical_outlier(nb_neighbors=50, std_ratio=2.0)
    pcd_clean = pcd.select_by_index(ind)

    # Transform to standard orientation
    # Flip Y for up-down, Flip Z for forward-back.
    # X is kept positive (1) to prevent mirroring.
    pcd_clean.transform([[1, 0, 0, 0], [0, -1, 0, 0], [0, 0, -1, 0], [0, 0, 0, 1]])
    return pcd_clean

# 1. RGB Point Cloud (standard)
pcd_rgb = create_pcd(cv_image, depth_map)
rgb_path = os.path.join(output_dir, "point_cloud_rgb.ply")
o3d.io.write_point_cloud(rgb_path, pcd_rgb)
print(f"✓ RGB point cloud: {rgb_path}")

# 2. Grayscale Point Cloud (alternative texture)
gray_image = np.stack([l_channel]*3, axis=-1) # Convert single channel to 3-channel
pcd_gray = create_pcd(gray_image, depth_map)
gray_path = os.path.join(output_dir, "point_cloud_gray.ply")
o3d.io.write_point_cloud(gray_path, pcd_gray)
print(f"✓ Grayscale point cloud: {gray_path}")

# 3. Heatmap Point Cloud (depth visualization)
# Technical Note: We use INFERNO colormap (not JET) because:
# - Perceptually uniform (equal color steps = equal depth steps)

```



```

# - Colorblind-friendly
# - Monotonically increasing luminance
# - Industry standard for scientific visualization
# Reference: "A Better Default Colormap for Matplotlib" (Smith & van der Walt, 2015)
depth_vis = cv2.applyColorMap((depth_map_norm * 255).astype(np.uint8), cv2.COLORMAP_INFERNO)
depth_vis_rgb = cv2.cvtColor(depth_vis, cv2.COLOR_BGR2RGB)
pcd_heatmap = create_pcd(depth_vis_rgb, depth_map)
heatmap_path = os.path.join(output_dir, "point_cloud_heatmap.ply")
o3d.io.write_point_cloud(heatmap_path, pcd_heatmap)
print(f"✓ Heatmap point cloud: {heatmap_path}")

# Save Raw Depth for Mesh Generation (Critical for textured mesh)
# This file is loaded by the mesh generation script
depth_npy_path = os.path.join(output_dir, "depth_map.npy")
np.save(depth_npy_path, depth_map_raw)
print(f"✓ Saved raw depth map: {depth_npy_path}")

print(f"\n✓ Generated 3 point cloud views from {image_path}")

if visualize:
    print("Visualizing RGB point cloud...")
    o3d.visualization.draw_geometries([pcd_rgb], window_name="RGB Point Cloud")

return rgb_path

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Convert 2D image to 3D point cloud")
    parser.add_argument("--image", type=str, default=os.path.join("assets", "demo03.jpg"), help="Path to input image")
    parser.add_argument("--focal-scale", type=float, default=0.8, help="Focal length scale factor")
    parser.add_argument("--depth-scale", type=float, default=1000.0, help="Depth scale factor")
    parser.add_argument("--visualize", action="store_true", help="Visualize result")

    args = parser.parse_args()

    image_path = args.image
    if not os.path.exists(image_path):
        print(f"Warning: {image_path} not found.")
        exit(1)

    process_depth_estimation(
        image_path,
        focal_length_scale=args.focal_scale,
        depth_scale=args.depth_scale,
        visualize=args.visualize
    )

```

Step 4 — Visualize Point Cloud

```
import open3d as o3d
import os
import sys

def visualize_point_cloud(ply_path):
    if not os.path.exists(ply_path):
        print(f"Error: Point cloud file not found at {ply_path}")
        return

    print(f"Loading point cloud from {ply_path}...")
    pcd = o3d.io.read_point_cloud(ply_path)

    num_points = len(pcd.points)
    print(f"✓ Loaded {num_points:,} points")
    print("\nVisualization Controls:")
    print("  - Left-click + drag: Rotate")
    print("  - Right-click + drag: Pan")
    print("  - Scroll: Zoom")
    print("  - Close window to exit")
    print("\nOpening visualization window...")

    # Create a visualizer object
    vis = o3d.visualization.Visualizer()
    vis.create_window(window_name=f"Point Cloud: {os.path.basename(ply_path)}", width=1280, height=720)
    vis.add_geometry(pcd)

    # Set a nice initial view
    vis.get_view_control().set_front([0, 0, -1])
    vis.get_view_control().set_lookat([0, 0, 0])
    vis.get_view_control().set_up([0, -1, 0])
    vis.get_view_control().set_zoom(0.8)

    vis.run()
    vis.destroy_window()
    print("Visualization closed.")

if __name__ == "__main__":
    # Check for command-line argument
    if len(sys.argv) > 1:
        ply_path = sys.argv[1]
    else:
        # Default to RGB point cloud
        ply_path = os.path.join("output", "point_cloud_rgb.ply")

    visualize_point_cloud(ply_path)
```

Step 5 — Mesh Generation

```
import open3d as o3d
import numpy as np
import os
import argparse

def create_mesh_from_points(pcd_path, output_path, depth=8, density_threshold=0.1, visualize=True):
    """
    Step 5: Basic Mesh Generation (Geometry Only)

    This script converts the Point Cloud into a solid mesh using Poisson Reconstruction.
    It focuses on GEOMETRY, while Step 6 focuses on TEXTURE.
    """

    if not os.path.exists(pcd_path):
        print(f"Error: File not found at {pcd_path}")
        return

    print(f"Loading Point Cloud: {pcd_path}")
    pcd = o3d.io.read_point_cloud(pcd_path)

    # Downsample if too large
    if len(pcd.points) > 500000:
        print("Downsampling for performance...")
        pcd = pcd.voxel_down_sample(voxel_size=0.01)

    # Orientation
    print("Estimating normals...")
    pcd.estimate_normals(search_param=o3d.geometry.KDTreeSearchParamHybrid(radius=0.1, max_nn=30))
    pcd.orient_normals_consistent_tangent_plane(k=10)

    # Poisson Reconstruction
    print(f"Running Poisson Reconstruction (depth={depth})...")
    mesh, densities = o3d.geometry.TriangleMesh.create_from_point_cloud_poisson(
        pcd, depth=depth, width=0, scale=1.1, linear_fit=False
    )

    # Cleanup
    print("Cleaning mesh artifacts...")
    densities = np.asarray(densities)
    vertices_to_remove = densities < np.quantile(densities, density_threshold)
    mesh.remove_vertices_by_mask(vertices_to_remove)

    # Save
    print(f"Saving mesh to {output_path}...")
    o3d.io.write_triangle_mesh(output_path, mesh)
    print("✓ Mesh saved")

    if visualize:
        print("Visualizing...")
        o3d.visualization.draw_geometries([mesh], window_name="Step 5: Geometry Mesh", mesh_show_back_face=True)

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--input", default=os.path.join("output", "point_cloud_rgb.ply"))
    parser.add_argument("--output", default=os.path.join("output", "model_mesh_geometry.obj"))
    parser.add_argument("--depth", type=int, default=8)
    parser.add_argument("--visualize", action="store_true")

    args = parser.parse_args()
    create_mesh_from_points(args.input, args.output, args.depth, visualize=args.visualize)
```

Step 6 — Textured Mesh

```
import open3d as o3d
import numpy as np
from PIL import Image
import os
import argparse
import shutil

def create_lightweight_textured_mesh(image_path, depth_map_path, output_path, depth_scale=500.0, downsample_factor=5):
    """
    Creates a Resource-Friendly Textured Mesh and packages it for viewing.
    """

    print(f"--- Low System Resource Mode ---")

    if not os.path.exists(depth_map_path) or not os.path.exists(image_path):
        print(f"Error: Missing input files.")
        return

    # 1. Load Data
    depth_map = np.load(depth_map_path)
    pil_img = Image.open(image_path)
    img_w, img_h = pil_img.size

    # Resize depth
    dh, dw = depth_map.shape[:2]
    if (dw, dh) != (img_w, img_h):
        depth_pil = Image.fromarray(depth_map)
        depth_pil = depth_pil.resize((img_w, img_h), Image.NEAREST)
        depth_map = np.array(depth_pil)

    # 2. Downsample Geometry
    w_vert = img_w // downsample_factor
    h_vert = img_h // downsample_factor

    print(f"Generating Mesh ({w_vert}x{h_vert})...")

    xx, yy = np.meshgrid(np.linspace(0, img_w-1, w_vert), np.linspace(0, img_h-1, h_vert))
    x_idx = np.round(xx).astype(int).clip(0, img_w-1)
    y_idx = np.round(yy).astype(int).clip(0, img_h-1)

    z_grid = depth_map[y_idx, x_idx]
    z_norm = (z_grid - z_grid.min()) / (z_grid.max() - z_grid.min())
    z_flat = z_norm.flatten() * depth_scale

    vertices = np.column_stack((xx.flatten(), yy.flatten(), z_flat))

    # 3. Triangles
    rows, cols = np.meshgrid(np.arange(h_vert-1), np.arange(w_vert-1), indexing='ij')
    v1 = rows * w_vert + cols
    v2 = v1 + 1
    v3 = v1 + w_vert
    v4 = v1 + w_vert + 1

    t1 = np.column_stack((v1.flatten(), v2.flatten(), v3.flatten()))
    t2 = np.column_stack((v2.flatten(), v4.flatten(), v3.flatten()))
    triangles = np.vstack((t1, t2))

    # 4. UVs
    u = xx.flatten() / (img_w - 1)
    v = 1.0 - (yy.flatten() / (img_h - 1))
    uvs = np.column_stack((u, v))
    tri_uvs = uvs[triangles.flatten()]
```

```

# 5. Build Mesh
mesh = o3d.geometry.TriangleMesh()
mesh.vertices = o3d.utility.Vector3dVector(vertices)
mesh.triangles = o3d.utility.Vector3iVector(triangles)
mesh.triangle_uvs = o3d.utility.Vector2dVector(tri_uvs)

# Transform
mesh.transform([[1, 0, 0, 0], [0, -1, 0, 0], [0, 0, -1, 0], [0, 0, 0, 1]])

# 6. Save OBJ
print(f"Saving to {output_path}...")
o3d.io.write_triangle_mesh(output_path, mesh)

# 7. Auto-Package for Viewing (Fix Material)
package_dir = "final_model"
if not os.path.exists(package_dir):
    os.makedirs(package_dir)

img_filename = os.path.basename(image_path)

# Write .mtl
mtl_content = f"""newmtl skin
Ka 1.000 1.000 1.000
Kd 1.000 1.000 1.000
d 1.0
illum 1
map_Kd {img_filename}
"""

with open(os.path.join(package_dir, "model.mtl"), "w") as f:
    f.write(mtl_content)

# Copy Image
shutil.copy(image_path, os.path.join(package_dir, img_filename))

# Fix OBJ and save to package
dst_obj = os.path.join(package_dir, "model.obj")

# We read the raw OBJ we just saved, and prepend the material link
with open(output_path, "r") as f_in:
    content = f_in.read()

with open(dst_obj, "w") as f_out:
    f_out.write("mtllib model.mtl\n")
    f_out.write("usemtl skin\n")
    f_out.write(content)

print(f"✓ Package ready in: {package_dir}")

# Visualize
print("\nVisualizing...")
mesh.textures = [o3d.geometry.Image(np.asarray(pil_img))]
mesh.compute_vertex_normals()
o3d.visualization.draw_geometries([mesh], window_name="Textured 3D Mesh")

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--image", default=os.path.join("assets", "demo04.jpg"))
    parser.add_argument("--depth", default=os.path.join("output", "depth_map.npy"))
    parser.add_argument("--output", default=os.path.join("output", "textured_mesh.obj"))
    parser.add_argument("--scale", type=float, default=500.0)
    parser.add_argument("--quality", type=int, default=5)

    args = parser.parse_args()
    create_lightweight_textured_mesh(args.image, args.depth, args.output, args.scale, args.quality)

```

Step 7 — Multi-View Fusion

```
import open3d as o3d
import numpy as np
import os
from transformers import pipeline
import cv2
from PIL import Image
import argparse
import copy

def preprocess_image(image_path):
    img = cv2.imread(image_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    return Image.fromarray(img), img

def get_depth_map(pipe, pil_image):
    result = pipe(pil_image)
    depth = np.array(result["depth"])
    return depth

def create_pcd_from_depth(rgb_image, depth_map, output_file=None):
    # Resize depth to match image
    h, w = rgb_image.shape[:2]
    d_h, d_w = depth_map.shape[:2]
    if (d_w, d_h) != (w, h):
        depth_map = cv2.resize(depth_map, (w, h))

    # Open3D structures
    o3d_color = o3d.geometry.Image(rgb_image)
    o3d_depth = o3d.geometry.Image(depth_map.astype(np.float32))

    # Intrinsic (focal length)
    focal_length = w * 0.8
    intrinsic = o3d.camera.PinholeCameraIntrinsic(w, h, focal_length, focal_length, w/2, h/2)

    # Generate RGBD
    rgbd = o3d.geometry.RGBDImage.create_from_color_and_depth(
        o3d_color, o3d_depth, depth_scale=1000.0, depth_trunc=1000.0, convert_rgb_to_intensity=False
    )

    # Generate Point Cloud
    pcd = o3d.geometry.PointCloud.create_from_rgbd_image(rgb, intrinsic)

    # Flip to be upright
    pcd.transform([[1, 0, 0, 0], [0, -1, 0, 0], [0, 0, -1, 0], [0, 0, 0, 1]])

    return pcd

def multi_view_fusion(image_folder, num_views, output_path="output/merged_model.ply"):
    """
    Stitches multiple images into a single 3D point cloud assuming a turntable rotation.
    """
    print(f"--- Multi-View Fusion (Turntable Mode) ---")

    # 1. Load Model
    print("Loading DepthAnything V2...")
    pipe = pipeline(task="depth-estimation", model="depth-anything/Depth-Anything-V2-Small-hf")

    # 2. Get Images
    # We look for all .jpg/.png in folder and sort them
    files = sorted([f for f in os.listdir(image_folder) if f.lower().endswith(('.jpg', '.png'))])
    # Limit to num_views if specified
    if num_views > 0:
```

```

files = files[:num_views]

if len(files) < 2:
    print("Error: Need at least 2 images for fusion.")
    return

print(f"Processing {len(files)} views...")

combined_pcd = o3d.geometry.PointCloud()

# 3. Process Each View
# We assume the object rotates 360 degrees over the sequence
angle_step = 360.0 / len(files)

for i, filename in enumerate(files):
    img_path = os.path.join(image_folder, filename)
    print(f"  View {i+1}/{len(files)}: {filename}")

    # A. Depth Estimation
    pil_img, cv_img = preprocess_image(img_path)
    depth_map = get_depth_map(pipe, pil_img)

    # B. Make Point Cloud
    pcd = create_pcd_from_depth(cv_img, depth_map)

    # C. Rotate (Align)
    # If object rotates on turntable, effectively the CAMERA rotates around it.
    # We rotate the point cloud back to a common frame.
    # Rotate around Y axis (standard Up)
    rotation_angle = np.radians(angle_step * i)

    # Create rotation matrix (Y-axis rotation)
    # [ cos  0  sin ]
    # [  0   1   0  ]
    # [-sin  0  cos ]
    R = pcd.get_rotation_matrix_from_xyz((0, rotation_angle, 0))

    # We rotate the cloud.
    # Note: Center of rotation is crucial. We assume object is centered at (0,0,Z).
    # Depth maps usually start at Z=0. We need to center the object first.
    center = pcd.get_center()
    pcd.translate(-center) # Move to origin
    pcd.rotate(R, center=(0,0,0)) # Rotate
    pcd.translate(center) # Move back? Actually for fusion we usually keep at origin.

    # Refinement: ICP (Iterative Closest Point)
    # If this isn't the first cloud, align it to the previous/combined one for tight fit
    if i > 0:
        print("    Aligning to previous view...")
        # Use point-to-point ICP for fine tuning
        threshold = 0.02
        trans_init = np.identity(4)
        reg_p2p = o3d.pipelines.registration.registration_icp(
            pcd, combined_pcd, threshold, trans_init,
            o3d.pipelines.registration.TransformationEstimationPointToPoint(),
            o3d.pipelines.registration.ICPConvergenceCriteria(max_iteration=30)
        )
        pcd.transform(reg_p2p.transformation)

    # Add to main cloud
    combined_pcd += pcd

# 4. Final Cleanup
print("Merging and cleaning...")
# Downsample to remove duplicate points from overlaps
combined_pcd = combined_pcd.voxel_down_sample(voxel_size=0.005)

```

```
# Statistical Outlier Removal (Clean noise)
cl, ind = combined_pcd.remove_statistical_outlier(nb_neighbors=20, std_ratio=2.0)
combined_pcd = combined_pcd.select_by_index(ind)

# 5. Save
o3d.io.write_point_cloud(output_path, combined_pcd)
print(f"✓ Saved merged model to: {output_path}")

# Visualize
o3d.visualization.draw_geometries([combined_pcd], window_name="Merged 3D Model")

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--folder", type=str, required=True, help="Folder containing image sequence")
    parser.add_argument("--views", type=int, default=0, help="Number of views to use (0=all)")
    parser.add_argument("--output", type=str, default="output/merged_model.ply")

    args = parser.parse_args()
    multi_view_fusion(args.folder, args.views, args.output)
```

Frontend — app.py

[illegible]

[illegible]

